



Cuestionario Teórico

Grupo 2

Técnicas Digitales III

Curso 512 - 2025

Iván Gonzalo Irumberry

Matwiejczuk, Leandro Darío

Profesor: Ing. MARÍN Ariel

Auxiliar: CARLASSARA Fabrizio

Auxiliar: VAN ROEY Julián

Parte Teórica: Modo Protegido

Realizar un informe desarrollando cada uno de los temas indicados a continuación:

1) Mecanismos de Protección de acceso al Segmento en Modo Protegido:

a. Realice un diagrama completo que permita comprender el mecanismo de protección de memoria del 80x386 que permite un no acceso directo al segmento. Explique detalladamente como interpreta un valor en un registro de segmento si TI=1 o TI=0.

En modo protegido los registros de segmento tienen la información para *ubicar* en la memoria del sistema *al descriptor* del segmento a acceder (**acceso no directo** al segmento), *en vez de contener toda la información* necesaria para definir el segmento (**acceso directo** al segmento) como en modo real.

Este acceso en forma indirecta es a través de “barreras de acceso”, y se implementa de la siguiente forma. Se tiene un **registro de segmento** de 2 bytes (16 bits en total) que contiene un campo selector de 13 bits (bits 15-3), 1 bit selector de tabla (bit 2), y un campo de nivel de privilegio solicitado (bits 1-0).



Bit 0-1: Denominado **RPL**, es el nivel de privilegio del selector. Admite 4 niveles de privilegios (00,01,10,11 del 0 al 3) donde 0 es el mayor nivel de privilegio.

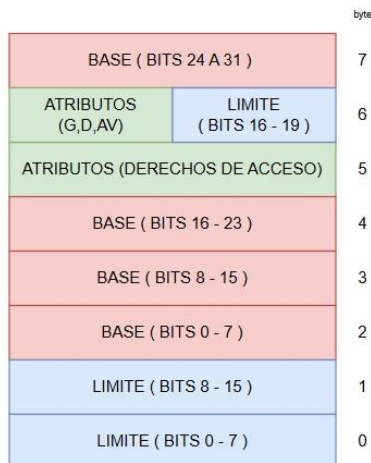
Bit 2: **TI** o Table Indicator, si TI = 0 se habla de una tabla de descriptor globales. Acá aparece la GDT. De lo contrario TI=1 es la tabla local. Esta GDT hay una sola a la cual todos los hilos de ejecución tienen acceso. Tablas de descriptors locales hay muchas, tantas como tareas (hilos de ejecución) se estén ejecutando. Cada tarea puede acceder a la GDT y a su propia tabla local. Pero no puede acceder a otras tablas locales.

Bits 3-15: EL índice (o **selector**) me permite elegir de todos los **descriptores** posibles (por ejemplo en TI=0) seleccionar 1 de todos los descriptors que se encuentran esa tabla de descriptors. Siendo 13 bits en total, hay una posibilidad de $2^{13} = 8192$ cantidad de descriptors.

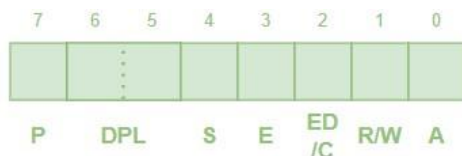
Y como se puede ver, a diferencia del modo real, no hay dirección efectiva ni nada de eso, **no hay de hecho un acceso directo**.

¿Y que es un **descriptor**? es una estructura de datos de 8 bytes que define (describe) en forma completa al segmento. Dicha estructura consta de la siguiente información:

La información mencionada en modo protegido, consta de un descriptor donde se va a definir la dirección a partir de la cual comienza el segmento en cuestión (Dirección Base), el tamaño del segmento (Limite), y los permisos de acceso al segmento como los de escritura/lectura, etc (Atributos).



Donde el byte 5 de Atributos está comprendido por los siguientes bits:



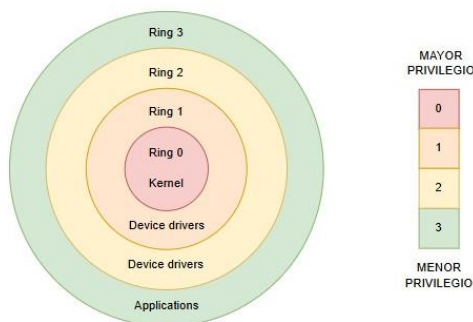
(La explicación de cada bit se detalla en el desarrollo del enunciado de los puntos a continuación)

b. ¿Cuáles son las condiciones que deben cumplirse para que el selector correspondiente pueda acceder al segmento? ¿Cuál es la relación entre RPL y DPL?

Para acceder al segmento, se llega mediante el índice RPL (Requestor Privilege Level) que apunta a un DPL (Descriptor Privilege Level) particular, uno dentro de una tabla global de descriptores. Dentro del descriptor en el campo de atributos tengo definido los 2 bits de DPL definiendo su nivel de privilegio (como se ha mostrado en la figura previamente). Se compara el valor de RPL de la tarea que generó inicialmente el selector que apunta al descriptor (y el segmento asociado con ese descriptor), también visto en la figura del registro de segmento.

La relación que se debe cumplirse es $RPL \geq DPL$ para que exista un acceso válido de ese selector a ese descriptor. Aquí se puede observar una barrera de acceso. Si ese selector no tiene un nivel de privilegio lo suficientemente alto, no va a poder acceder al segmento.

Respecto a los posibles valores, aquí se consideran de la siguiente forma:



El 0 es el más alto como se puede ver en la imagen ilustrativa. No es del valor absoluto del número matemáticamente, es a valor “lógico”, como se ve en los anillos de privilegios.

Por ejemplo, un RPL=0 puede acceder a un segmento cuyo descriptor tiene un DPL=2.

c. Explique que son y para qué sirven los siguientes registros y bits: GDTR, GDTR, RPL, LDTR, bit G, bit D, bit P, bit S, bit E, bit A.

GDTR: No se encontró tal registro en los apuntes de clase, lo más cercano a esa nomenclatura es una instrucción “LGDT” que carga la base y el límite de la tabla de descriptores global en el registro apropiado.

GDTR: Registro que mantiene la dirección base de 14 bits y el límite de 16 bits de la tabla GDT. Es un segmento global para todas las tareas del sistema.

RPL: Campo de 2 bits (el 0 y 1 del registro de segmento) que define el nivel de privilegio de la tarea solicitante.

LDTR: Registro que mantiene los selectores de 16 bits para el descriptor LDT. Este segmento es específico para tarea y apunta a un descriptor apropiado de LDT.

Bit G: Es el bit de “granularidad” que se encuentra dentro de los atributos en un descriptor dado. Si $G=0$ el límite especifica un límite de segmento de 00000H a FFFFFH. Si $G=1$, el valor del límite sería de 00000FFFFH a FFFFFFFFH. Esto permite una longitud de segmento de 4K a 4G bytes según la arquitectura de 16bits o de 32bits (Ej sistema operativo DOS o WIN32).

Bit D: Es el bit “Default”, también dentro del campo de atributos en el mismo descriptor, es el bit que indica como las instrucciones del microprocesador acceden a los datos de los registros y de la memoria en el modo real o protegido. Si $D=0$ instrucciones de 16 bits (modo DOS). Si $D=1$ las instrucciones son de 32 bits.

Bit P: Es el bit “Presente”, dentro del campo de atributos del descriptor va a establecer $P=0$ si el descriptor está indefinido o $P=1$ si el segmento contiene una base y límite válidos (existe en memoria física).

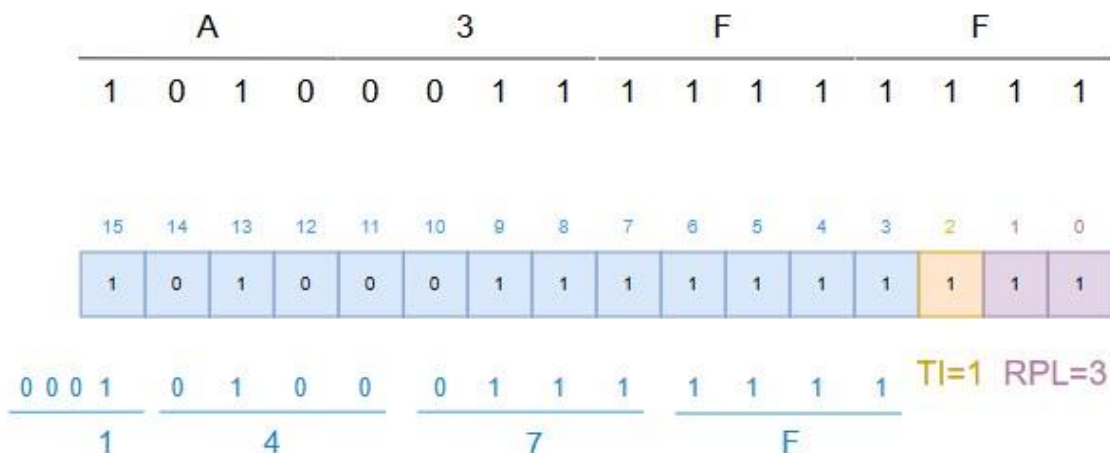
Bit E: Es el bit “Ejecutable” número 3 dentro de los atributos del descriptor donde su valor corresponde a 1 para segmento de código y a 0 para segmento de datos.

Bit S: Bit “Descriptor de segmento”, dentro del campo de atributos del descriptor y vale $S=1$ para descriptores de código y datos, y $S=0$ para descriptores del sistema.

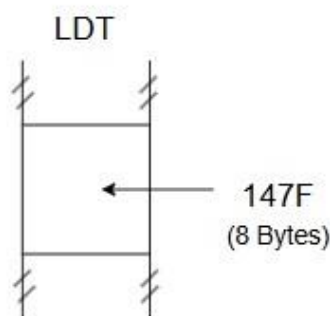
Bit A: es el bit 0 del campo de atributos del descriptor y su valor es A=0 para cuando no se ha accedido previamente al segmento, mientras que A=1 cuando se ha accedido al segmento.

d. Práctica: Ejercicio de MP (Interpretación y funcionamiento) Consigna: Realice un diagrama interpretando en Modo Protegido el siguiente selector: CS=A3FF.

Sea el selector: CS=A3FF



LO QUE QUIERE DECIR ESTE REGISTRO DE SEGMENTO ES QUE EL DESCRIPTOR EN CUESTIÓN SE ENCUENTRA EN LA ENTRADA 147F EN LA TABLA LDT



2) Descriptores de Segmento y Descriptores de Sistema:

a. ¿Cuántas tablas de descriptores existen en la arquitectura 80x386? Explique cada una de ellas. NOTA: No es lo mismo "tablas de descriptores" que "descriptores".

Las tablas de descriptores definen los segmentos utilizados en el sistema. Hay tres tipos de tablas: la Tabla de Descriptores Globales (GDT), la Tabla de Descriptores Locales (LDT) y la Tabla de Descriptores de Interrupción (IDT).

GDT: Contiene descriptores que están disponibles para todas las tareas. Puede contener cualquier clase de descriptores de segmento excepto los relacionados con interrupciones. Contiene los segmentos de código y datos usados por el sistema operativo, los TSS (es un descriptor de segmento de estado de la tarea y se detalla futuramente en ítem punto del enunciado) y los descriptores para las LDT en un sistema.

LDT: Contiene descriptores asociados con una tarea particular. Solo puede contener descriptores asociados a código, datos, pila, compuertas de tarea y compuertas de llamada. Cada tarea tiene su propia LDT y proveen un mecanismo para aislar los segmentos de código y datos de una tarea respecto de otras. Esto permite el aislamiento y protección de los segmentos de una tarea, mientras que los datos comunes pueden ser compartidos por todas las tareas.

IDT: Contiene los descriptores que apuntan a la ubicación de hasta 256 rutinas de servicio de interrupción (interrupt handlers). El número es 256 porque es la cantidad de tipos de interrupciones diferentes que maneja el micro. No se puede definir en la IDT un descriptor de segmento de datos ni de código. Se trata de descriptores de sistema (Bit S=0 en el descriptor). Por lo que los 3 tipos de descriptores posibles en la IDT son: Interrupt Gate, Trap Gate, Task Gate.

b. Explique de qué forma actúa el bit S como atributo de un descriptor. Utilice esta información para decir en qué tipo de descriptor se convierte el mismo. ¿Qué “tipos” de estos descriptores existen?

El bit S es el bit 4 del byte número 5 dentro de la estructura de un descriptor, correspondiente al campo de atributos. Si S=1 el segmento es de código o datos y la estructura del descriptor es como la demostrada previamente. Ahora si S=0, el formato del byte de derechos de acceso para descriptores de segmentos tiene los siguientes cambios:



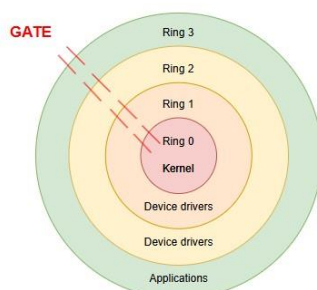
Los bits 3-0 definen tipos de descriptor, por ejemplo 0100 Compuerta de llamada, 0101 Compuerta de tarea, 0110 Compuerta de Interrupcion, 0111 Compuerta de trampa, etc.

c. Explique el concepto y funcionamiento de un “Call via Gate”.

En relación a lo previamente mencionado con el caso del bit S=0, se tiene entonces descriptores de sistema. Dentro de sus tipos, están los descriptores de LDT (caso 0010), los descriptores de TSS (caso 0001 y 0011) y a lo que respecta la pregunta, los descriptores de compuerta (tipos 4 a 7, como se puede ver en su valor binario).

Estas compuertas se utilizan para controlar el acceso a puntos de entrada dentro del segmento de código objetivo. Los distintos tipos de compuerta son: **de llamada (call gate)**, de tarea (task gate), de interrupción (interrupt gate) y de trampa (trap gate).

Específicamente la compuerta de llamada se utilizan para cambiar niveles de privilegio. Gráficamente:



Estas GATES proveen llamadas indirectas, en forma protegida. Uno de los usos más importantes de las compuertas es poder tener un método seguro de transferencia de privilegio dentro de una tarea. Como el sistema operativo define todas las compuertas en el sistema, puede asegurar que todas las compuertas permiten el acceso sólo a los procedimientos confiables.

3) Segmento de Estado de la Tarea (TSS):

a. ¿Qué se busca con el uso del TSS relacionado al uso del procesador y la conmutación de tareas?

Respecto a la habilidad para cambiar entre tareas en un sistema operativo multitarea, para esta operación se utiliza el descriptor TSS que apunta a un segmento que contiene el estado de la ejecución del sistema. Para conmutar de tarea, una instrucción (JMP o CALL) se ejecuta para referirse a un segmento de estado de la tarea TSS y se busca salvar el estado entero de la máquina (todos los registros, espacio de direcciones, y un puntero a la tarea anterior), para luego cargar el nuevo estado de ejecución, verificaciones de protección y comenzar la ejecución. De esta forma, lo que se obtiene son tareas ejecutándose “al mismo tiempo” donde en realidad el procesador ejecuta una a la vez por un tiempo determinado dando una “sensación de multitarea en paralelo”.

b. Explique el funcionamiento y realice un diagrama de interpretación del mismo. Esquema del TSS.

31	15	0	
I/O Map Base Address	Reserved	T	100
Reserved	LDT Segment Selector		96
Reserved	GS		92
Reserved	FS		88
Reserved	DS		84
Reserved	SS		80
Reserved	CS		76
Reserved	ES		72
	EDI		68
	ESI		64
	EBP		60
	ESP		56
	EBX		52
	EDX		48
	ECX		44
	EAX		40
	EFLAGS		36
	EIP		32
	CR3 (PDBR)		28
Reserved	SS2		24
	ESP2		20
Reserved	SS1		16
	ESP1		12
Reserved	SS0		8
	ESP0		4
Reserved	Previous Task Link		0

Es el lugar de memoria previsto en los preprocesadores IA-32 como espacio de contexto de cada tarea. El tamaño mínimo de este segmento es de 67h. Se guardan los valores de SS Stack Segment (registro de la pila) y ESP (Registro extendido en 32 bits del Stack Pointer) para los stacks de nivel 2,1 y 0 (los diferentes anillos previamente vistos). El del nivel 3 es una tarea que se va a ejecutando en usuario, eventualmente estará en la dirección lógica SS:ESP (Registro de segmento: Desplazamiento de ese registro de segmento). El I/O MAP Base Address es un mapa de permisos de entrada y salida que va a tener la tarea para majear los diferentes tipos de puertos que tiene la tarea.

En resumen, son todos los registros que se guardan para analizar el contexto de una tarea.

Para su funcionamiento, cada TSS necesita de un descriptor que va a estar en la GDT y allí se busca. Donde un registro de cache oculto se conforma con el TR (Registro de Tarea) y entre ellos se permite direccionar al TSS, y en función de ese descriptor TSS, el segmento que describe en forma completa al estado de esa tarea que fue almacenado en memoria.

4) Paginado:

a. Explique conceptualmente para qué sirve el proceso de paginado y como se relaciona con el sistema de segmentado.

La paginación es un tipo de manejo de memoria útil para sistemas operativos multitarea que manejan memoria virtual. A diferencia de la segmentación que modulariza programas y datos en segmentos de longitud variable, la paginación divide los programas en varias páginas de tamaño fijo. Estas páginas no tienen ninguna relación con la estructura lógica del programa.

b. ¿Qué hace la unidad de paginado? ¿Cómo lo hace?

Entre el procesador y la memoria tenemos una unidad de segmentado. Entre la unidad de segmentado y la memoria hay una nueva unidad, la de paginación. Básicamente cumple la función de traducir direcciones virtuales generadas por el programa en ejecución hacia direcciones físicas.

en la memoria RAM utilizando una tabla de páginas. La cual es mantenida por el SO y mapea las páginas de memoria virtual y las ubicaciones correspondientes en la memoria física.